

Techniques défensives et offensives des applications Web · Module 8

Les bonnes pratiques de sécurité, par catégorie

Des **ressources OWASP** et de l'**ASVS niveau 2** à une **contre-mesure de référence** pour chacune des dix catégories du Top 10:2025.



Objectifs pédagogiques

À l'issue, vous saurez :

- Exploiter les **ressources OWASP** (Top 10, WSTG, Cheat Sheets, WebGoat...).
- Situer l'**ASVS** (version 5.0) et viser un **niveau 2**.
- Associer à chaque catégorie sa **contre-mesure de référence**.
- Transformer ces pratiques en **exigences vérifiables** et en contrôles.

Le changement d'angle

Les modules 5-6 montraient **quoi corriger**. Ici : **comment structurer** la défense — un **standard**, des **preuves** (ASVS), des **ressources**, une **culture**.

Fil rouge

Pour chaque catégorie : **la pratique** → l'**exigence ASVS** qui la vérifie → l'**outil** qui l'automatise.

Du « quoi corriger » au « comment garantir »



L'idée

Une bonne pratique qui n'est ni **écrite** (standard), ni **vérifiée** (ASVS / revue), ni **automatisée** (CI) ne tient pas dans la durée. Ce module relie les trois.

La suite

Le **module 9** approfondit les mécanismes de protection (CSRF, CORS, CSP, HSTS...) et le **module 10** l'automatisation (SAST/DAST/SCA, quality gates).

01

L'écosystème OWASP

Des ressources ouvertes, gratuites, pour concevoir, coder, tester et s'entraîner.

Dans cette partie

- Panorama des ressources
- Top 10 et WSTG
- Code Review et Cheat Sheets
- WebGoat et Juice Shop
- Proactive Controls et SAMM
- API Security Top 10

La bonne ressource pour chaque besoin



Top 10

Les risques majeurs — sensibilisation



ASVS

Exigences vérifiables — le « quoi prouver »



WSTG

Testing Guide — comment tester



Code Review Guide

Revue de code orientée sécurité



Cheat Sheets

Fiches pratiques par sujet



Proactive Controls

Top 10 côté développeur



WebGoat / Juice Shop

S'entraîner légalement



SAMM

Mesurer la maturité

À retenir

Le Top 10 **sensibilise**, l'ASVS **exige**, le WSTG **teste**, les Cheat Sheets **guident le code**, WebGoat/Juice Shop **entraînent**, SAMM **mesure**.

Top 10 et Web Security Testing Guide (WSTG)

OWASP Top 10

Le classement des 10 risques — excellent pour **sensibiliser** et prioriser, mais ce n'est **pas** une checklist de test exhaustive.

Édition 2025

Rappel : l'ordre a changé (Misconfiguration en A02, Supply Chain en A03, nouvelle catégorie A10 Mishandling of Exceptional Conditions).

WSTG — Web Security Testing Guide

Un **catalogue méthodique** de tests (chaque test a un identifiant **WSTG-xxxx**) : authentification, session, entrées, logique métier, API...

Sert de **plan de test** reproductible pour un audit ou un pentest interne.

Code Review Guide et Cheat Sheet Series

Code Review Guide

Comment **relire du code** avec un œil sécurité : où regarder, quels motifs dangereux, comment prioriser.

✓ CHEAT SHEETS — LES INCONTOURNABLES

- Authentication · Password Storage · Session Management
- SQL Injection Prevention · XSS Prevention · Output Encoding
- Access Control · REST Security · Secrets Management

Usage concret

Chaque Cheat Sheet donne des **règles directement applicables** et du code. On les référence dans son **standard de codage** interne et dans les revues : « ce point suit la Cheat Sheet Password Storage ? ».

WebGoat et OWASP Juice Shop



WebGoat

Application **pédagogique** guidée : des leçons expliquent chaque faille, on l'exploite puis on la corrige. Idéal pour **apprendre**.



Juice Shop

Application **réaliste volontairement vulnérable**, avec un **score-board** de défis. Idéal pour **s'exercer** et animer des CTF internes.

Cadre

Ces cibles s'exécutent en **environnement isolé** (Docker local), jamais exposées. Excellents supports de **formation** et de **sensibilisation** des équipes de développement (lien module 7).

Proactive Controls et SAMM

OWASP Proactive Controls

Le pendant **développeur** du Top 10 : les contrôles à mettre en place **positivement** (définir les exigences, requêtes paramétrées, encoder, contrôler l'accès, gérer l'authentification, protéger les données, journaliser...).

SAMM — Software Assurance Maturity Model

Un modèle pour **évaluer et faire progresser** la maturité sécurité d'une organisation (gouvernance, conception, implémentation, vérification, opérations) — par niveaux (vu au module 2).

✓ **COMMENT S'EN SERVIR** Proactive Controls = base d'un **standard de codage**. SAMM = **feuille de route** d'amélioration (où en est-on, quelle prochaine étape).

Les 10 contrôles côté développeur

- **C1** — Définir les exigences de sécurité
- **C2** — S'appuyer sur des frameworks et bibliothèques sûrs
- **C3** — Sécuriser l'accès aux **bases de données** (requêtes paramétrées)
- **C4** — **Encoder** et échapper les données
- **C5** — **Valider** toutes les entrées
- **C6** — Gérer **identité et authentification**
- **C7** — Appliquer le **contrôle d'accès**
- **C8** — Protéger les **données** (au repos et en transit)
- **C9** — **Journaliser** et surveiller
- **C10** — Gérer les **erreurs** et exceptions

À quoi ça sert

Là où le Top 10 liste les **risques**, les Proactive Controls listent les **bonnes actions** — la base idéale d'un standard de codage. Notez C10 (gestion des erreurs), écho direct à la nouvelle catégorie A10:2025.

OWASP API Security Top 10

Pourquoi un Top 10 à part

Les **API** ont des risques spécifiques, mal couverts par le Top 10 « web » : autorisation au niveau **objet** et **fonction**, exposition excessive de données, consommation de ressources.

Réflexe

Pour une appli riche en API : croiser le **Top 10 web**, l'**API Security Top 10** et le **chapitre API de l'ASVS**.

Risques emblématiques

- **BOLA** — autorisation objet cassée (IDOR d'API, n°1)
- **BFLA** — autorisation fonction cassée
- Exposition excessive de données / propriétés
- Consommation de ressources non limitée
- Inventaire d'API incomplet (shadow / zombie APIs)

02

L'ASVS et son niveau 2

Transformer « être sécurisé » en exigences précises et vérifiables.

Dans cette partie

- Un référentiel d'exigences vérifiables
- Les niveaux L1, L2, L3
- Exemples d'exigences L2
- Trois usages concrets
- D'une exigence à sa preuve

Un référentiel d'exigences vérifiables

Définition

L'**ASVS** (Application Security Verification Standard) est une liste d'**exigences de sécurité vérifiables**, organisée par **domaines**. Chaque exigence est **testable** — on peut prouver qu'elle est satisfaite.

Version 5.0 (mai 2025)

Refonte majeure : ~**350 exigences** réparties en **17 chapitres**, orientée SDLC et cloud/API, avec des identifiants traçables (**5.0.0-x.y.z**). Les trois niveaux L1/L2/L3 sont conservés.

Domaines couverts (extrait)

- Contrôle d'accès
- Authentification
- Gestion de session
- Jetons (JWT, OAuth/OIDC)
- Validation et encodage
- Cryptographie
- Communications (TLS)
- Configuration
- Journalisation et erreurs
- Code malveillant / intégrité
- Fichiers et ressources
- API et services web
- Protection des données
- Sécurité front-end

L1, L2, L3 — viser le bon

Niveau 1

Sécurité de **base**, résistance aux attaques courantes. Minimum pour toute application.

Niveau 2 ★

La **cible de la plupart des applications** traitant des données sensibles (métier, personnelles, financières). Le **standard best practices**.

Niveau 3

Applications **critiques** (santé, défense, forte valeur). Exigences les plus strictes ; vérification qui exige l'accès aux artefacts internes.

En pratique

La formation vise le **niveau 2** : c'est le standard attendu d'une application d'entreprise sérieuse. On **déclare** son niveau cible et on le **vérifie**. En 5.0, le niveau 3 insiste sur le fait que le test en boîte noire seul ne suffit pas.

À quoi ressemble une exigence vérifiable (esprit L2)

Domaine	Exemple d'exigence (esprit L2)
Authentification	Mots de passe $\geq$ 12 caractères, vérifiés contre une liste de mots de passe compromis
Contrôle d'accès	Refus par défaut ; chaque accès à une ressource vérifie l'autorisation côté serveur
Validation / Encodage	Requêtes paramétrées systématiques ; sortie encodée selon le contexte
Cryptographie	Algorithmes approuvés, clés gérées, aléa cryptographique
Journalisation	Événements de sécurité journalisés, sans données sensibles
Fichiers / SSRF	Accès aux URL externes en liste blanche , IP résolue validée

Ce qui change

« Vérifier l'autorisation » devient une **ligne cochable**, testée, tracée — pas un vœu pieux.

Trois usages concrets



Checklist de revue

Passer chaque fonctionnalité au crible des exigences du niveau visé.



Plan de test

Dériver les cas de test (couplé au WSTG) et les rejouer à chaque version.



Clause contractuelle

Exiger d'un prestataire une conformité **ASVS L2**, vérifiable et opposable.

🛡️ **DEFINITION OF DONE SÉCURITÉ** Intégrer une sélection d'exigences ASVS dans le « **terminé** » d'une user story : une fonctionnalité n'est livrée que si ses exigences de sécurité sont satisfaites.

D'une exigence à sa preuve



Le cycle vertueux

Une exigence ASVS n'est pas un slogan : elle se **traduit en code**, se **vérifie par un test** rejoué à chaque build, et laisse une **trace**. C'est ce qui rend la conformité **démontrable** (audit, client, régulateur).

03

Une contre-mesure par catégorie

Pour chaque risque du Top 10:2025 : la pratique de référence, le domaine ASVS, l'outil.

Dans cette partie

- La grille A01 → A10
- A01-A05 : accès, config, chaîne, crypto, injection
- A06-A10 : conception, auth, intégrité, logs, erreurs

A01 → A10 (édition 2025), d'un coup d'œil

Catégorie 2025	Contre-mesure de référence	Domaine ASVS
A01 Broken Access Control	Refus par défaut, contrôle serveur, requêtes scopées (+ anti-SSRF)	Contrôle d'accès
A02 Security Misconfiguration	Configuration minimale et reproductible	Configuration
A03 Supply Chain Failures	Inventaire (SBOM), veille, signature	Intégrité
A04 Cryptographic Failures	Algorithmes approuvés, secrets et clés gérés	Cryptographie
A05 Injection	Requêtes paramétrées, encodage contextuel	Validation
A06 Insecure Design	Conception sécurisée, threat modeling	Conception
A07 Authentication Failures	Stockage des mots de passe, anti-brute-force, MFA	Authentification
A08 Integrity Failures	Signature et vérification d'intégrité	Intégrité
A09 Logging & Alerting	Journalisation exploitable et alertes	Journalisation
A10 Exceptional Conditions	Fail-closed, gestion d'erreurs, atomicité	Gestion d'erreurs

Contrôle serveur, refus par défaut (+ anti-SSRF)

- ✓ Tout refuser par défaut, autoriser explicitement
- ✓ Autorisation **centralisée**, jamais dispersée
- ✓ Vérifier la **propriété** de l'objet (requête scopée)
- ✓ DTO en entrée (anti mass assignment)
- ✓ **Anti-SSRF** : allowlist des destinations, IP résolue validée
- ✓ Tester : chaque rôle × chaque endpoint

```
http.authorizeHttpRequests(a -> a
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()); // deny by default
@PreAuthorize("hasRole('ADMIN')") // method security
Facture get(Long id, User u){
    return repo.findByIdAndOwnerId(id,u.id())...; }
```

Configuration minimale et reproductible

- ✓ Durcissement **par défaut**, surface minimale
- ✓ Config **reproductible** (Infrastructure as Code)
- ✓ Profils **dev** / **prod** séparés
- ✓ En-têtes de sécurité, CORS restreint, erreurs génériques
- ✓ Scanner la config (Trivy, kube-bench, Checkov)

```
# scanner une config IaC
checkov -d ./infra
trivy config ./k8s
# en-têtes (Spring Security 6)
http.headers(h -> h.frameOptions(f->f.deny()))
    .httpStrictTransportSecurity(...);
```

Inventaire des dépendances (SBOM), veille

- ✓ **SBOM** généré à chaque build (CycloneDX)
- ✓ Versions **verrouillées** (lockfile / BOM), inutiles retirées
- ✓ **SCA** en CI, seuil bloquant (HIGH/CRITICAL)
- ✓ Mises à jour **automatisées** (Dependabot / Renovate)
- ✓ Artefacts **signés** (cosign), provenance (SLSA)

```
mvn cyclonedx:makeAggregateBom # SBOM
trivy fs --exit-code 1 \
  --severity HIGH,CRITICAL . # bloque la CI
mvn org.owasp:dependency-check-maven:check
```

Algorithmes approuvés, secrets et clés gérés

- ✓ Primitives éprouvées : **AES-GCM**, **Argon2**, TLS 1.2+
- ✓ Jamais de crypto « maison », jamais de clé en dur
- ✓ Clés dans un **KMS / coffre**, rotation planifiée
- ✓ Aléa **cryptographique** (**SecureRandom**)
- ✓ Préparer la migration **post-quantique** (données longue durée)

```
# secrets injectés à l'exécution, jamais dans Git
export DB_PASSWORD=$(vault kv get -field=pwd secret/db)
# détecter les fuites de secrets en CI
gitleaks detect --source .
```

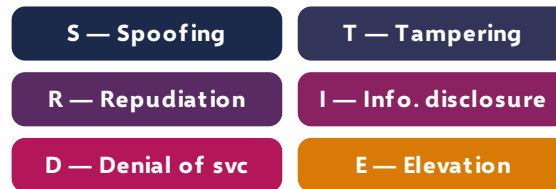
✓ **COFFRE** Vault / AWS Secrets Manager, rotation automatique, audit d'accès aux secrets.

Requêtes paramétrées, encodage contextuel

- ✓ **Bind** systématique (SQL, LDAP, NoSQL)
- ✓ Allowlist pour les identifiants non liables (ORDER BY)
- ✓ **Encodage en sortie** selon le contexte (HTML/JS/URL)
- ✓ **CSP** stricte en défense en profondeur
- ✓ API sans shell pour les commandes système

```
// requête paramétrée
jdbc.query("... WHERE name = :n", Map.of("n",name), m);
// sortie encodée (Thymeleaf)
<div th:text="${comment}"></div>
// commande sans shell
new ProcessBuilder("ping","-c","1",host);
```

Conception sécurisée et threat modeling



un crible pour chaque composant et flux

- ✓ Modéliser les menaces **avant de coder** (schéma de flux, frontières de confiance)
- ✓ Passer chaque composant au crible de **STRIDE** (module 2)
- ✓ Définir **limites, quotas, états** et cas d'abus
- ✓ Réutiliser des **patterns sûrs** (Proactive Controls, ASVS)

Mots de passe, anti-brute-force, MFA

- ✓ Hash **lent et salé** (Argon2id / bcrypt)
- ✓ Mots de passe longs, vérifiés contre les fuites connues
- ✓ **Limitation** des tentatives + verrouillage
- ✓ **MFA** (idéalement WebAuthn / FIDO2 — passkeys)
- ✓ Sessions régénérées, jetons courts et validés

```
var enc = new Argon2PasswordEncoder(16,32,1,1<=16,3);
if (enc.matches(pwd, hash)           // temps constant
    && limiter.tryConsume(ip)) {      // anti brute-force
    request.changeSessionId();        // anti-fixation
    requireMfa(user);
}
```

Signature et vérification d'intégrité

- ✓ **Signer** et **vérifier** artefacts et images (cosign)
- ✓ **SRI** pour les scripts tiers
- ✓ Allowlist de classes en désérialisation
- ✓ Pipeline CI/CD durci, provenance (SLSA)

```
cosign sign registry/monapp:1.4.2
cosign verify registry/monapp:1.4.2 \
  --certificate-identity ci@mycorp
<script src="https://cdn/lib.js"
  integrity="sha384-..." crossorigin></script>
```

Journalisation exploitable et alertes

- ✓ Journaliser les **événements de sécurité** (échecs, refus)
- ✓ **Jamais** de secrets / données personnelles dans les logs
- ✓ Entrées **assainies** (anti log injection, CRLF)
- ✓ **Centraliser** (SIEM), corrélér, **alerter** au seuil
- ✓ Protéger l'intégrité des journaux

```
log.warn("auth_failure user={} ip={} reason={}",  
        sanitize(userId), ip, "bad_credentials");  
// règle d'alerte SIEM  
count(auth_failure) BY ip > 20 / 1min → alerte
```

Fail-closed, gestion des erreurs, atomicité

- ✓ Un contrôle qui **échoue** doit **refuser** (fail-closed), jamais autoriser
- ✓ Gérer **toutes** les exceptions, ne rien avaler
- ✓ Opérations **atomiques** / idempotentes (anti-TOCTOU)
- ✓ Messages d'erreur **génériques** (détails dans les logs)
- ✓ Tester les **chemins d'erreur**, pas seulement le cas nominal

```
try { return pdp.check(u, r); }  
catch (Exception e) {  
    log.error("authz_error", e);  
    return false; // erreur → on REFUSE (fail-closed)  
}  
// débit atomique : UPDATE ... WHERE balance >= :a
```

04

Ancrer les bonnes pratiques

Standard écrit, revue de code, culture — pour que ça tienne dans la durée.

Dans cette partie

- Un standard de codage sécurisé
- La revue de code sécurisée
- De la pratique au contrôle continu

Un standard de codage sécurisé

Le principe

Rassembler les règles (Proactive Controls, Cheat Sheets, exigences ASVS L2) dans un **document vivant**, versionné, connu de toute l'équipe.

Il devient la référence des **revues** et de la **Definition of Done**.

✓ EXEMPLES DE RÈGLES

- Toute requête SQL est **paramétrée**
- Toute sortie est **encodée** selon son contexte
- Refus par défaut sur chaque endpoint
- Aucun secret dans le code (vérifié par Gitleaks)

La revue de code sécurisée

Où regarder en priorité

- Points d'**entrée** (paramètres, uploads, désérialisation)
- Contrôles d'**accès** et d'authentification
- Requêtes **SQL** et sorties **HTML**
- Gestion des **secrets** et de la crypto
- **Chemins d'erreur** (fail-open ?)

✓ BONNES PRATIQUES DE REVUE

- Un **Security Champion** dans la boucle (module 3)
- SAST en **appui** (pré-filtre), pas en remplacement
- Checklist ASVS pour les fonctions sensibles
- Revue **sans blâme**, on corrige le processus

De la bonne pratique au contrôle continu



SAST

Analyse du code — Semgrep, SonarQube



SCA

Dépendances et SBOM — Trivy, Dependency-Check



DAST

Appli en exécution — OWASP ZAP



Secrets

Fuites de secrets — Gitleaks



IaC

Config — Checkov, Trivy



Gate

Seuil bloquant — Quality Gate

Le lien

Chaque bonne pratique devient un **contrôle automatique** dans le pipeline (module 3). Le **module 10** détaille cette automatisation (SAST/DAST/SCA, quality gates).

La gestion des secrets

- ✓ **Zéro secret** dans le code ou dans Git
- ✓ Centralisés dans un **coffre** (Vault, Secrets Manager)
- ✓ **Injectés à l'exécution** (variables d'environnement)
- ✓ **Rotation** régulière, accès **audité**
- ✓ Détection en CI (**Gitleaks**, TruffleHog)

```
# récupérer un secret au démarrage
export DB_PASSWORD=$(vault kv get \
  -field=pwd secret/prod/db)
# bloquer un push contenant un secret
gitleaks protect --staged --redact
```

🛡️ **UN SECRET QUI A FUITÉ EST COMPROMIS** Le **révoquer** et le **régénérer**, ne pas juste le retirer de l'historique.

Definition of Done et quality gate

Definition of Done « sécurité »

Une user story n'est **terminée** que si ses critères de sécurité sont satisfaits — pas « on verra plus tard ».

- ✓ Exigences ASVS de la fonctionnalité vérifiées
- ✓ Revue de code sécurité passée
- ✓ Aucune vulnérabilité HIGH/CRITICAL ouverte

Quality gate (CI)

La porte automatique qui **bloque** la livraison si les seuils ne sont pas tenus.

```
# le pipeline échoue → pas de livraison
trivy fs --exit-code 1 --severity HIGH,CRITICAL .
sonar: bloquer si new_vulnerabilities > 0
```

La checklist de l'application sûre

- ✓ Refus par défaut, contrôle d'accès serveur (+ anti-SSRF)
- ✓ Config minimale et reproductible
- ✓ Dépendances inventoriées (SBOM), signées et à jour
- ✓ Crypto approuvée, secrets en coffre
- ✓ Requêtes paramétrées, encodage en sortie
- ✓ Conception modélisée (menaces, cas d'abus)
- ✓ Auth robuste : Argon2, MFA, anti-brute-force
- ✓ Artefacts signés et vérifiés (intégrité)
- ✓ Journaux exploitables + alertes
- ✓ Fail-closed, gestion des erreurs et conditions limites

Les sigles du module en un coup d'œil

OWASP — Open Worldwide Application Security Project

SAMM — Software Assurance Maturity Model

DoD — Definition of Done (critères de « terminé »)

SBOM — Software Bill of Materials (inventaire)

SSRF — Server-Side Request Forgery (→ A01 en 2025)

TOCTOU — condition de course (check puis use)

ZAP — OWASP Zed Attack Proxy (DAST)

ASVS — Application Security Verification Standard (v5.0)

Cheat Sheet — fiche de bonnes pratiques par sujet

Quality Gate — seuil de sécurité bloquant en CI

KMS — Key Management Service (gestion des clés)

STRIDE — méthode de threat modeling (6 menaces)

IaC — Infrastructure as Code

BOLA / BFLA — autorisation objet / fonction cassée (API)

WSTG — Web Security Testing Guide

Proactive Controls — 10 contrôles côté développeur

SAST / DAST / SCA — analyse statique / dynamique / dépendances

MFA — Multi-Factor Authentication

Fail-closed — en cas d'erreur, refuser l'accès

SIEM — corrélation de journaux et alertes

SRI / SLSA — intégrité des scripts / provenance des artefacts

Ce qu'il faut retenir

- L'**OWASP** fournit tout : sensibiliser, exiger, tester, guider, entraîner.
- L'**ASVS L2** (v5.0) transforme la sécurité en **exigences vérifiables**.
- Chaque catégorie a une **contre-mesure de référence** — à écrire, vérifier, automatiser.
- Une pratique ni écrite, ni vérifiée, ni automatisée ne **tient pas dans la durée**.

Et ensuite ?

Le **module 9** approfondit les **mécanismes de protection** (CSRF, cookies, CORS, CSP, HSTS, API) et le **module 10** l'**automatisation** (SAST/DAST/SCA, quality gates).

L'idée à emporter

La sécurité durable = un **standard** écrit, des **preuves** (ASVS), des **contrôles** automatiques, une **culture**. Les quatre ensemble.

Sources et références

Ressources OWASP

- **Top 10:2025, ASVS 5.0** (asvs.dev), **WSTG**
- **Proactive Controls, Cheat Sheet Series**
- **API Security Top 10, SAMM**
- **WebGoat, Juice Shop** (entraînement)

Outils cités

- Semgrep, SonarQube, Trivy, Dependency-Check
- OWASP ZAP, Gitleaks, Checkov, cosign, Vault

Points à actualiser

L'**ASVS** est passé en **version 5.0** (mai 2025) : les numéros de chapitre diffèrent de la 4.0.3 — ici les domaines sont nommés par thème. Le **Top 10** est réédité tous les 3-4 ans.

Cohérence

Ce module suit l'édition **2025** du Top 10, comme les modules 5, 6 et 7. Les contre-mesures A01-A10 y sont réordonnées en conséquence.

Fin du module 8

Des questions ?

Des ressources OWASP à l'ASVS L2, chaque bonne pratique devient une **exigence vérifiable** et un **contrôle**. Place aux mécanismes de protection et à l'automatisation.